Workload Management: DIRAC v9 and the way to DiracX

Alexandre F. Boyer · CERN

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026



One year after the last workshop

What the DUW11 WMS talk said would come next

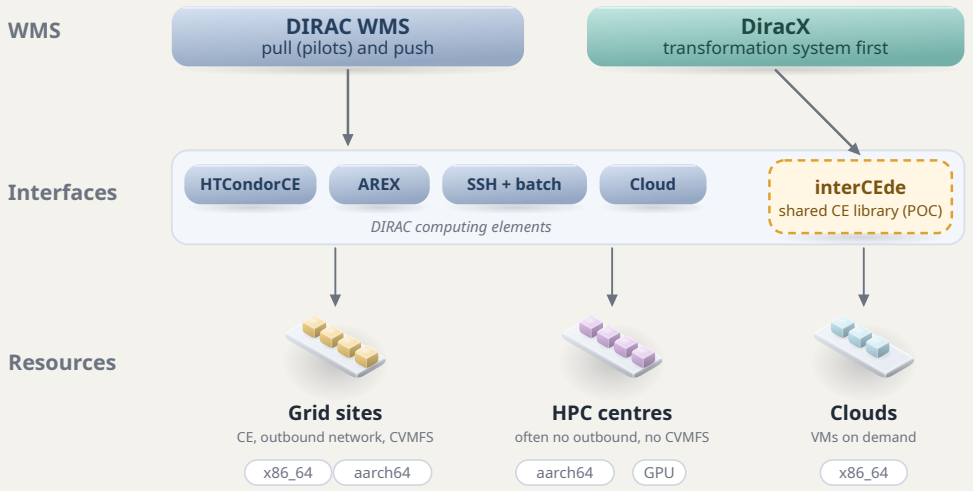
ANNOUNCED IN 2025	WHERE IT IS	
SSH computing element on Fabric	**done**	DIRAC #7703, v9.0
CPU and memory limits with cgroup2	**done**	#8110, plus memory-aware matching (#8366, #8414)
Python 2 out of the pilot	**done**	Pilot #268, Nov 2025
Multi-architecture images from CVMFS	**pull request open**	DIRAC #8456, to certify
Pilots managed by DiracX	**started, then paused**	pilot database and router in DiracX (#570)
CWL job submission in DiracX	**redirected**	now part of the transformation system ADRs
Job monitoring and job manager in DiracX	**stopped**	no central JobDB in DiracX: section 4
A decision on DB12	**open**	DB12 #15, #16

Most of what was planned for DIRAC landed. The DiracX part changed direction, for a reason explained in section 4.

1 · Resources and interfaces

What we run on, and how we reach it

Resources keep diversifying



- Grid, HPC and clouds, now with **aarch64** and GPUs beside x86_64.
- HPC centres often have no outbound network and no CVMFS.
- DIRAC v9 dropped the CREAM, ARC, ARC6 and BOINC CEs; AREX is the ARC interface, with tokens.
- One computing element interface per DIRAC installation today: no sharing with other WMSs.

interCEde: one library for computing elements

A typed, asynchronous library to submit to and manage computing elements, usable by DIRAC, DiracX and other workload managers. github.com/DIRACGrid/intercede, proof of concept

Design

Three ADRs under review (intercede #4): computing elements, integration tests, credentials.

Open points: per-site submit options, whole-node and GPU requests, ARC delegation renewal, cleaning up cloud VMs.

Code

Core contract proposed this week (#27).

Integration-test scaffolding (#25), and test setups from the communities: HTCondor, ARC with Slurm, Slurm.

Next

Merge the ADRs, then: core, registry, integration tests, backends.

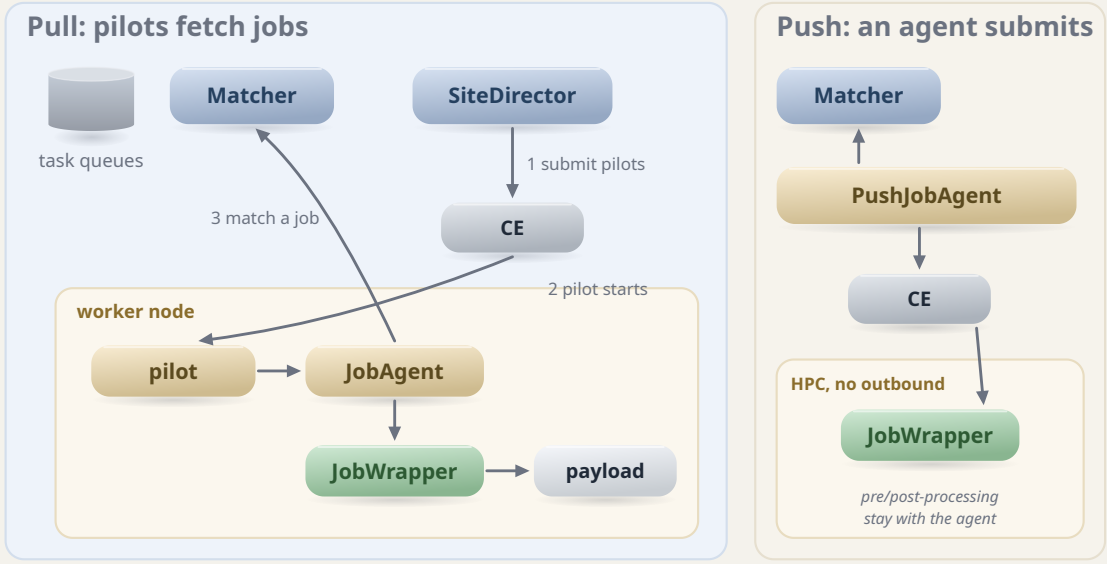
Last, a synchronous facade and an adapter so DIRAC can use it (#24).

Contributions from several communities; the HTCondor grid universe through a local schedd is deferred, not adopted.

2 · The DIRAC WMS since v9

Faster, more reliable, and the same two models

Two models, unchanged



- **Pull**: one SiteDirector per VO, CEs handled in parallel.
- **Push**: for nodes that cannot reach DIRAC; pre- and post-processing stay with the agent.
- The changes since v9 are inside the boxes: the task queues, the Matcher, the JobAgent and the JobWrapper.

Faster

MEASURED ON PRODUCTION-SIZED DATABASES	BEFORE	AFTER	RELEASE
Task queue matching query	~30 ms	**~3 ms**	v9.1.0 · #8462
Finding matching task queues (323 queues)	3.4 s	**0.2 s**	v9.1.20 · #8801
Job insertion	~115 jobs/s	**~190 jobs/s**	v9.1.20 · #8801
Duplicate task queues during a burst	64	**1**	v9.1.20 · #8801
Delay between matches, per job type	1 s or more	**sub-second**	v9.1.16 · #8711

- Also: missing indices on pilots and jobs (#8463, manual `ALTER TABLE`), hot paths found with py-spy (#8524), fewer calls in the pilot agents (#8569, #8624).

> Several fixes made the DIRAC WMS faster and more reliable, for installations at v9 and above. Most of the numbers above need v9.1.

More reliable, safer, simpler

Reliable

- Task queues no longer lose their site, platform or tag restrictions under concurrency (#8801).
- Memory-aware matching; cgroup2 limits in containers (#8366, #8414, #8110).
- Banned CEs skipped; rescheduled jobs not failed; no NaN parameters (#8340, #8441, #8750).

Safer

- SQL parameterised in every WMS database (v9.1.13 to v9.1.15).
- Five WMS advisories fixed: job state access, sandbox store, pilot logging, pilot manager, pilot download.
- Patched in v9.1.20, v9.0.26, v8.0.85.

Simpler

- Removed: CREAM, ARC, ARC6, BOINC CEs; pilot bundles; private pilots; Python 2 in the pilot.
- Tokens: pilots submitted with tokens in multi-VO setups; proxy pilots also carry one.

→ *DIRAC is patched, not developed:* **upgrading is how you get these**

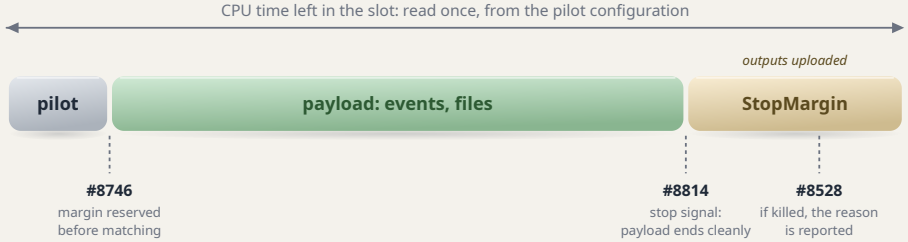
Time left in the slot

The problem

- Jobs in containers could not ask the batch system for the time left.
- The post-processing margin was known only to the Watchdog, which could only kill.

An example: an MC job sized for 183 events produced 150, was killed, and its output was never uploaded.

Since v9.1.4 (#8479): the pilot reads the time left once; jobs read it from the pilot configuration.



The three fixes above are in **v9.1.21** only. Set the `StopSig*` options in the CS or per job; see "Allocated time management" in the Administrator Guide.

Multi-architecture: one image name, every architecture

DIRAC #8456 (pull request open)

- The pilot detects the node's architecture and resolves the container image under the CVMFS `.multiarch` layout: `<base>/<arch>/<image>`.
- Used by the Singularity CE and `dirac-apptainer-exec`; falls back to `ContainerRoot`.
- Left to do: certify against `unpacked.cern.ch`, review, merge.

What a community needs

- Images published as multi-architecture (OCI index) images; otherwise the layout has dangling links.
- Platform tags in the pilot's architecture script.

Not solved yet: a job that can run on x86_64 *or* aarch64 has a single set of requirements in today's matcher. Section 3.

> The same job description on any architecture, without per-architecture configuration.

3 · Proofs of concept

Matching, and the pilot

A matcher for DiracX

Goal (diracx #843): match pilots to jobs at **1 kHz** (200 Hz at least), with 10 million waiting jobs per VO. Redis holds an index that can be rebuilt from the database.

Built: matching with tag expressions, prioritisation, a benchmark harness, a Redis data model (release 0.4.0, Sep 2026).

Measured: the pure Python baseline only, 210 ms per match at 50k candidate jobs, 5.7 s at 750k. No Redis numbers yet.

What we learned

- Real requirements vary little: LHCb has 872k waiting jobs but 14 distinct (CPU time, group) pairs. Group jobs by profile, as task queues do; do not test them one by one.
- A job eligible at ~49 sites must not be stored 49 times.
- A job runnable on two architectures needs a *list* of requirements.

Next: matching inside Redis (Lua) and its benchmark on realistic data (#873), then scale tests (#874). **Then an ADR**, written from those numbers, for the matcher of the `diracx-pilot` backend.

Pilots: the re-thinking has not started

Landed

- A pilot database and router in DiracX (#570, Aug 2026).
- Pilot: Python 2 removed; maintenance fixes.

Paused

- Pilot registration and authentication with pilot tokens (#421).
- Pilot logging through DiracX (#550).
- Untouched since mid-2025.

Questions for later

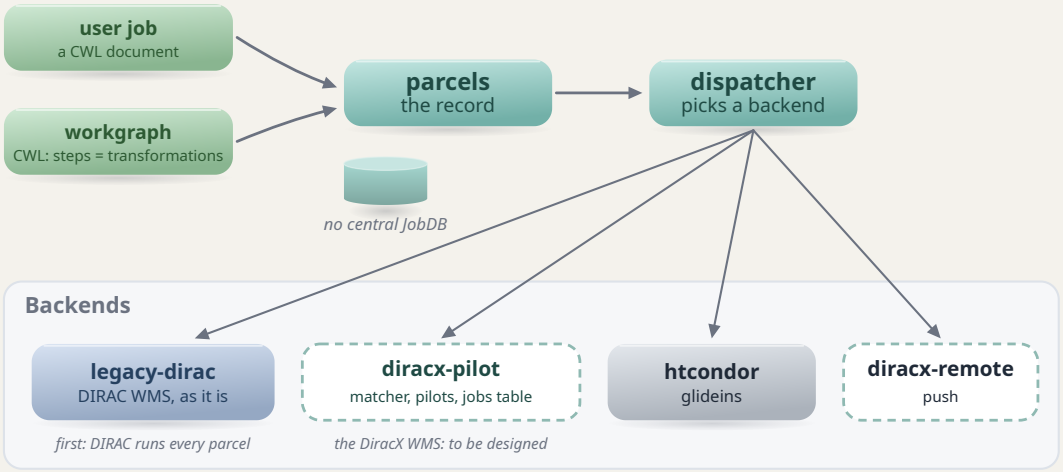
- What does a pilot need to know about parcels?
- What belongs to the pilot, what to the job wrapper?
- One model for pull and push?

> Pilots belong to the DiracX WMS backend, which comes after the transformation system. We continue to patch the DIRAC pilot.

4 · DiracX: the WMS follows the transformation system

No JobDB at the centre, CWL everywhere

Work is a parcel, dispatched to a backend



- DiracX has no "Systems". The transformation system ADRs cover productions, transformations **and the job part of the WMS**: user jobs are parcels too.
- First, every parcel goes to DIRAC. A native WMS backend comes when the transformation system needs it: **there will be a WMS plan**, not yet.

No JobDB at the centre

In DIRAC, the JobDB was the first database, and every system reads or writes it.

In DiracX, the record of work is the **parcel**. A jobs table, if any, would be private to the WMS backend (`diracx-pilot`), like any backend's own state.

The JobDB already in DiracX stays, with a narrow use:

- job status updates from DIRAC, through the future client;
- load-testing a new installation, and checking it is deployed correctly.

We do not port the rest of job management (monitoring, manager) to it.

> The DiracX WMS will evolve slowly, in parallel, until the transformation system needs a native backend.

CWL describes all the work

One format (DX-ADR-007, draft): a user job, a workgraph and each transformation are CWL v1.2 documents.

- DiracX settings are **hints** under `dirac:`, never requirements: every document stays plain, runnable CWL.
- The same document runs with `cwltool`, locally against grid data, and at scale.
- Processes are resolved before dispatch: the node runs without fetching definitions.

What this means for the WMS

- Matching requirements come from the hints.
- The job wrapper resolves inputs (`lfn:`, `sandbox:`), captures the output sandbox, and writes a status report per input.
- Outputs are registered by DiracX after the parcel ends, not by the wrapper.

Today: `dirac-cwl` 1.3.0; an experimental CWL job wrapper in DIRAC (#8425).

The job wrapper: one for all of us, owners wanted

Today there are several

- DIRAC's JobWrapper: pre-processing, payload, post-processing; an offline variant for push.
- The `dirac-cwl` wrapper, experimental in DIRAC.
- Executor prototypes in DiracX.

Each community also adds its own steps around the payload.

Requirements to gather

- Inputs: data, sandboxes, protocol access.
- Time left, graceful stop, containers, architectures.
- Nodes without outbound network.
- Reporting: status, parameters, logs.
- What happens when an upload fails.

We are looking for owners: people who collect the requirements from the communities, set priorities, and accept the result, in a product-owner role. Come and talk to us this week.

Take-aways

FOR INSTALLATIONS

- Upgrade to v9.1: faster matching and job insertion, time-left fixes (v9.1.21), security patches.
- Multi-architecture images: #8456, soon.

PROOFS OF CONCEPT

- Matcher: Redis numbers next, then an ADR.
- interCEde: ADRs close to merge, core in progress.
- Pilots: re-thinking not started.

FOR DIRACX

- The transformation system comes first; the WMS follows when it is needed.
- No JobDB at the centre: work is a parcel.
- CWL describes everything that runs.

WE NEED YOU

- Owners for the job wrapper: requirements, priorities, acceptance.

Thank you

Questions and comments welcome, this week and after.

Alexandre F. Boyer · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org

